

TITLE AND DATE

Document name: Project#4 PWM Motor Driver Lab Report

Document reference: kiddKid.cmpe311.fall25.project#4

Date of publication: December 2, 2025

LEAD ENGINEER: Jessica Slaughter, UMBC

STAKEHOLDERS:

Prof Kidd, Instructor, University of Maryland Baltimore County, Baltimore, Maryland, USA

MD Safwan Zaman, TA, University of Maryland Baltimore County, Baltimore, Maryland, USA

HIGH-LEVEL DESCRIPTION: This document is the project report for Project #4 in CMPE311. The project involves the design of an embedded system that allows a motor's speed to be controlled by the repetitive pressing of a button. It includes customer, technical, and testing requirements, as well as the design details and validation test results.

DESCRIPTION: This design is to create a circuit on an Arduino Uno R3 development platform that allows the user of the program to control the speed of a motor with a push button. Included in this document are the customer requirements obtained from the customer, the high-level technical requirements derived from those customer requirements, the design, the testing scenarios and requirements, and the results of testing. Appended to this document is the code executed and a video of those tests that required video documentation.

RESULT SUMMARY: The project was a success, the embedded system design meeting all testing and high-level requirements.

REFERENCES AND GLOSSARY

REFERENCES:

- ProjectAsyncTasking_draft – The definition of the project this document addresses
- CMPE310 Project #5 – A similar project assigned in CMPE310 in Spring 2025
- Arduino UNO R3 Product Reference Manual SKU A000066, 12/03/2024
- Async Programming in Arduino: Unleashing the Power of Non-Blocking Code, Mahdi Valizadeh, medium.com, 4/8/2024

DEFINITIONS:

“The User” – The person operating (not programming) the embedded system

“The System” – The embedded system being operated by The User

“The Customer” – The person(s) paying for the embedded system being designed and built

“The Developer” – The person(s) designing and building the System

“The Evaluator” – The person(s) that determine whether or not The System satisfies The Customer-requirements.

“The Customer-requirements” – The requirements defined by The Customer as satisfying The Contract.

“The Requirements” – The System’s high-level technical requirements derived from The Customer-requirements.

“The Educational-constraints” – Requirements imposed by the instructor unrelated to the embedded system that allow The System to be evaluated.

“The Company” – The organization The Customer has contracted with to build The System.

“The Contract” – The business document that legally binds The Company to provide some service or product to The Customer.

“serial-monitor” – The serial port used by the Arduino IDE to communicate with The User.

“The Reference-platform” – The configuration of The System used by The Developer to test and validate The System. For this class, The System is the Arduino compatible ELEGOO Uno R3 development board.

“PROJECT-ASYNC” – The previous project upon which this project is based

ACRONYMS AND ABBREVIATIONS:

Arduino – an Italian open-source hardware and software company; also refers to a development board created by the company

arduino.h – header for a library of convenience functions specific to the Arduino development platform

AVR – A family of microcontrollers, originally developed by Atmel, and currently owned by Microchip Technology

ELEGOO – A Chinese company that develops and markets 3D printers and accessories

IDE – Integrated Development Environment

gcc – front end for the GNU Compiler Collection

Github – A widely used distributed SVC (Software Version Control) system

LED – Light Emitting Diode

REQUIREMENTS

CONVENTIONS:

Must, shall or will – your design must satisfy the requirement

May – your design may satisfy the requirement but doesn't have to

Informative – the intent of the following description is to make the requirement more understandable

All customer requirements are started with "C.#".

All high-level requirements are started with "HL.#".

All testing/validation requirements are started with "T.#"

CUSTOMER REQUIREMENTS:

- C1. The User must be able to set the blink rate of two different LEDs.
- C2. The User must be able to update the blink rate of each of the LEDs independently.
- C3. The LED must blink at the set rate until The User tells the LED to blink at a different rate.
- C4. The System must run upon an Arduino Uno R3 compatible development board.
- C5. The blink rate of an LED must be expressed in terms of milliseconds.
- C6. The motor must be controlled by the button.
- C7. The system must allow the motor to cycle through at least 5 distinct speed levels, including off.
- C8. The system must operate asynchronously with other tasks (e.g., LED blinking).

HIGH-LEVEL TECHNICAL REQUIREMENTS:

- HL.1. The User through the IDE serial monitor must be able to set the blink rate of two different LEDs.
 - HL.1.1. The blink rate of each LED must be able to be set independent of the other LED.
 - HL.1.2. The setting of an LED blink rate must not interfere with the blinking of the LEDs until the new LED and blink rate are specified. (*From requirements C2 and C3*)
- HL.2. The user through the IDE serial monitor must set the blink rate in terms of once every N milliseconds (*From requirement C5*)
- HL.3. The System must run upon an Arduino Uno R3 compatible development board.
- HL.4. PWM signal for motor control must be generated using a hardware timer and ISR.
- HL.5. PWM duty cycle must be adjustable in steps
- HL.6. Button input must be software-debounced to prevent false triggering.
- HL.7. The system must maintain correct operation of LEDs and motor control tasks concurrently (asynchronous task scheduling).
- HL.8. The motor driver circuit must include a MOSFET and flyback diode to handle inductive load safely.
- HL.9. The motor driver must allow safe operation with the specified motor voltage (3–12V) and current (0.25–3.5A).

EDUCATIONAL REQUIREMENTS:

E.1.0 Implement a cyclic executive task manager to dispatch the asynchronous tasks you created in the previous project, PROJECT-ASYNC.

E.1.1 The system testing and validation requirements must contain those defined in PROJECT-ASYNC.

E.1.1.1 The testing and validation requirements must contain any additional tests necessary to properly evaluate/demonstrate the function of the system.

E.1.2 The final report must be a formal design document as in PROJECT-ASYNC

E.1.3. Include a schematic and parts list for the motor driver circuit.

E.1.4. Include calculations for current-limiting and pull-down resistors in the report.

E.1.5. Demonstrate and explain PWM waveform and duty cycle adjustment for motor speed.

E.1.6. Validate that motor speed control is synchronized with button presses and operates asynchronously with other tasks.

DESIGN:

DESIGN PRE-REQUISITES:

1. ELEGOO Arduino Uno R3 clone
2. Arduino IDE 2.3.3 or better
3. Motor
4. Push button
5. MOSFET

DEVELOPMENT PLATFORM:

1. See DESIGN PRE-REQUISITES above

ANY ADDITIONAL DESIGN CONSIDERATIONS:

1. None

See Appendix A, B for the video of the code operating on the Arduino Uno R3.

See Appendix D, C for the C code.

CALCULATIONS:

PROBLEM #1 — Calculate R1

$$R_1 = V(I) = 5 \text{ V} (10 \text{ mA}) = 500 \text{ ohms}$$

R1 protects the Arduino from excessive current if the MOSFET gate fails short. Normal operation does not use significant current. So for this application, I chose the closest resistor value I had: 470 ohms.

PROBLEM #2 — Constraints for R2

I selected 10k Ω because it fully pulls the MOSFET gate to ground without significantly loading the Arduino pin.

PROBLEM #3 — Minimum Response Time of ATmega328P @ 16MHz

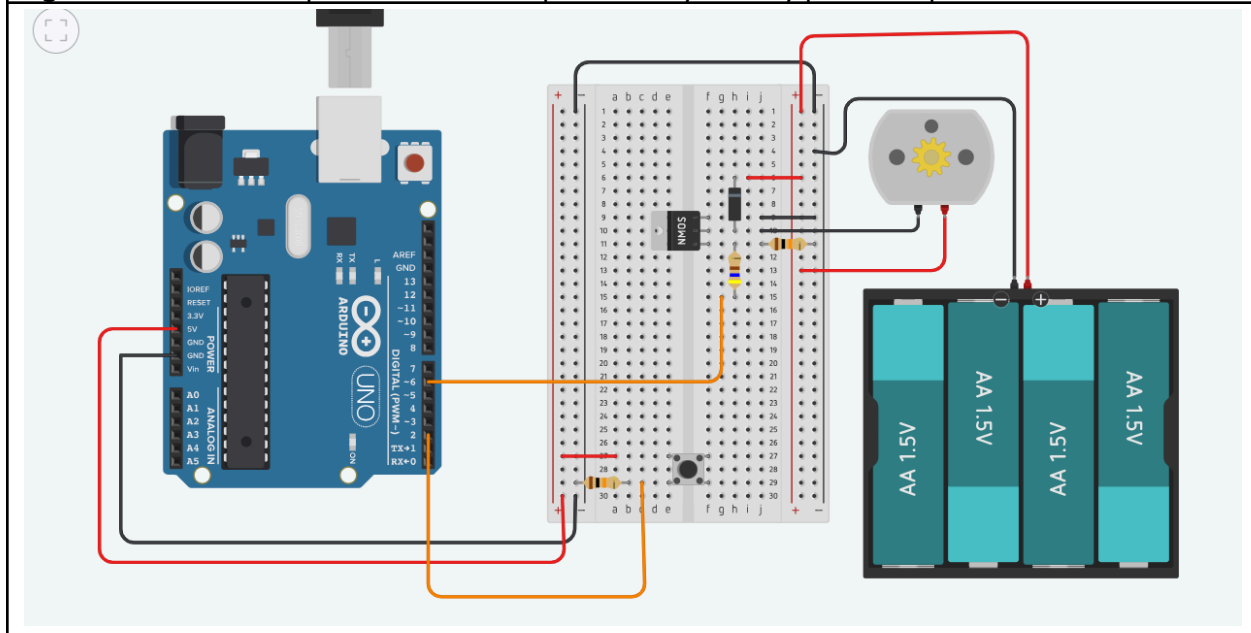
$$T = 1/16\text{MHz} = 62.5 \text{ ns}$$

The theoretical minimum response time of the ATmega328p running at 16 MHz is 62.5 nanoseconds, equivalent to one clock cycle.

TESTING AND VALIDATION

DESCRIPTION: The tests that have to be performed and validated are shown in Table 2. These tests were performed on the testbed shown Figure 1. Table 1 shows a dialog that must be successfully performed by the embedded system. The results of testing are shown in Table 3. A video of the test is provided along with this report.

Figure 1. Testbed Setup. MOSFET motor powered by battery pack and push button.



TESTING PLATFORM:

1. ELEGOO Arduino Uno R3 clone
2. Arduino IDE 2.3.3 or better
3. Power: Testing platform powered through USB cable, LEDs connected directly through Digital Outputs
4. Circuit elements: breadboard, LEDs, resistors, push button, motor, battery pack, and wires

Figure A. Operation of project circuit		
Action user preforms	What happens to the motor	Output of Serial Monitor
(Initial state)	*motor is off*	(Not specified and up to you)
Push button	motor operates at 2/8 duty cycle	
Push button	motor operates at 4/8 duty cycle	
Push button	motor operates at 6/8 duty cycle	
Push button	motor operates at 8/8 duty cycle	
Push button	motor operates at 6/8 duty cycle	
Push button	motor operates at 4/8 duty cycle	
Push button	motor operates at 2/8 duty cycle	
Push button	*motor is off*	

Figure B. Simple motor driver using a N-Channel MOSFET.

Note: modify the source voltage to match the voltage of the motor, 6Vdc.

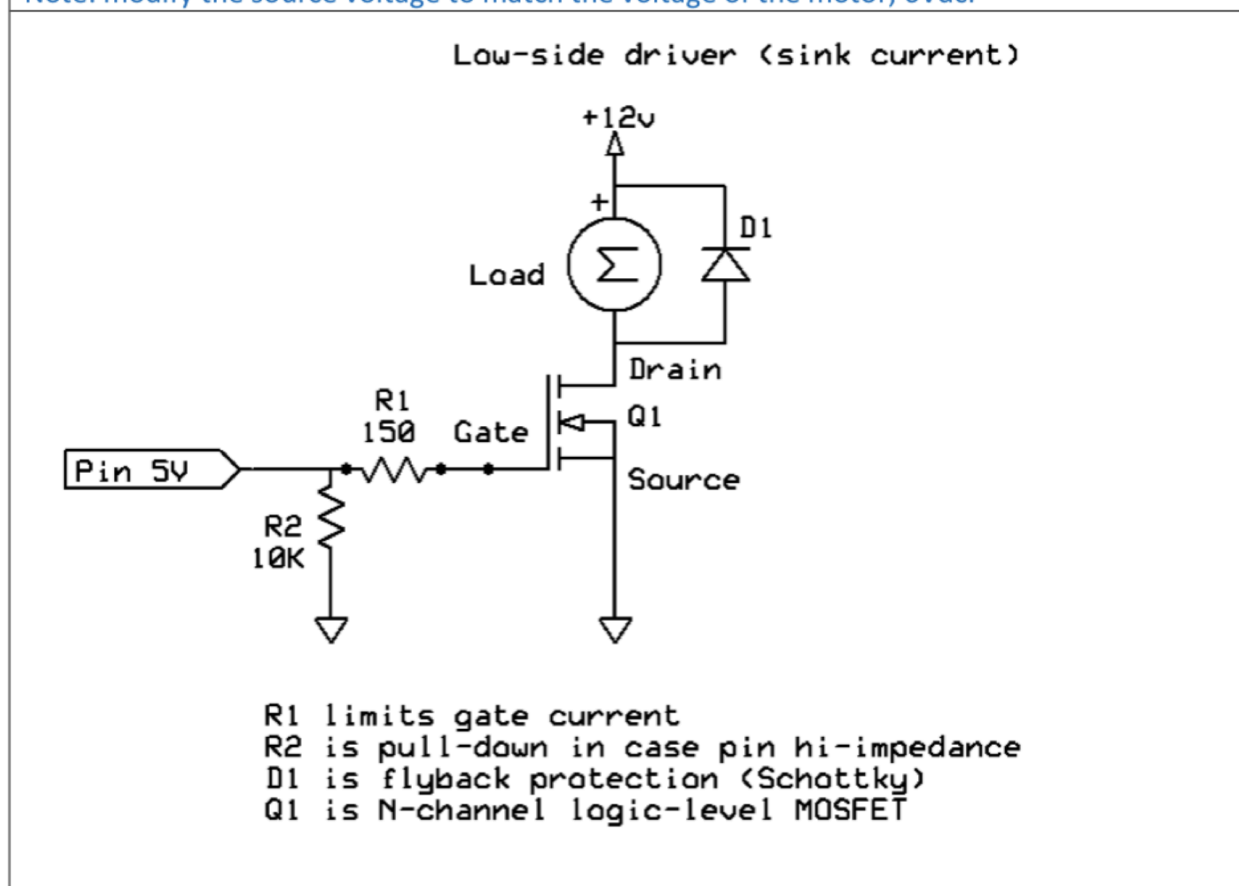


Table A. DC Motor Specifications

Voltage: 3V to 12V
 Speed: 1500 to 6500 RPM
 No-load current: 0.25 Amps
 Max Efficiency current: 0.64 Amps
 Stall current: 3.5 Amps

TESTING AND VALIDATION REQUIREMENTS:

Table 2. Testing and Validation Requirements

T.0	All testing and validation must be done on the testbed illustrated in Figure 1.
T.1	The dialog above (or similar) must work as shown
T.1A	The blink rate of an LED must be able to be set without interfering with the blinking of the other LED
T.1.1	Setting of the blink rate (and LED#) must be through the IDE serial monitor
T.2	The blink rate of the LED being set must not change until the input is complete
T.3	The user must specify the blink rate in milliseconds per blink
T.4	The blink rate specified by the user must be correctly reflected on the testbed LEDs.
T.4.1	The blink rate specified by the user must be reflected on the LED specified by the user
T.5.	The setting of an LED's blink rate must be able to be repeated at least 5 times
T.5.1.	Successively for the same LED
T.5.2.	Alternately between the different LEDs

TESTING RESULTS:

Table 3. Results of tests

Test performed	Results
T.1.1	Satisfied. See video of test.
T.1	Satisfied
T.2	Satisfied. See video of test.
T.3	Satisfied
T.4	Satisfied
T.4.1	Satisfied. See video of test.
T.5.1	Satisfied. See video of test.
T.5.2	Satisfied. See video of test.

APPENDIX A. VIDEO WITH LED

Google drive link to video of code running on circuit with led with pwm control:

https://drive.google.com/file/d/1W_cLq0dfatc8tKi2bPXwjISgYguz8eCT/view?usp=sharing

APPENDIX B. VIDEO WITH MOTOR

Google drive link to video of code running on circuit with motor with pwm control:

<https://drive.google.com/file/d/18TbgYzBuiXiDbGoGcode6qqrujeQ-9Rv/view?usp=sharing>

APPENDIX C. PART I DESIGN CODE

Link to GitHub: https://github.com/jesslaughter/CMPE311_F25

```
//PROJECT: PWM Motor Driver (Part I)
// Pin assignments
#define LED_PIN1 2
#define LED_PIN2 3
#define BTN_PIN 4
#define PWM_PIN 9

#include <Arduino.h>

// CE LED
unsigned long interval1 = 0;
unsigned long interval2 = 0;
unsigned long prevMillis1 = 0;
unsigned long prevMillis2 = 0;

bool state1 = LOW;
bool state2 = LOW;

int step = 0;
int chosen_led = 0;

//PWM
volatile uint8_t pwmDuty = 0;
volatile uint8_t brightnessIndex = 0;

// 5 brightness steps
const uint8_t brightnessSteps[6] = {0, 1, 2, 3, 4, 5};
int direction = 1;

// Debounce
unsigned long lastDebounce = 0;
```

```

const unsigned long debounceDelay = 40;
bool lastReading = HIGH;
bool stableState = HIGH;

// Task Manager
void TaskToggleLED1();
void TaskToggleLED2();
void TaskSerialInput();
void TaskPWMBUTTON();

typedef void (*TaskFn)();
TaskFn tasks[] = {TaskToggleLED1, TaskToggleLED2, TaskSerialInput, TaskPWMBUTTON};
const uint8_t NUM_TASKS = 4;

void setup() {
    Serial.begin(9600);
    // CE LED
    pinMode(LED_PIN1, OUTPUT);
    pinMode(LED_PIN2, OUTPUT);
    //PWM BUTTON
    pinMode(BTN_PIN, INPUT_PULLUP);
    pinMode(PWM_PIN, OUTPUT);

    digitalWrite(LED_PIN1, LOW);
    digitalWrite(LED_PIN2, LOW);

    noInterrupts();
    TCCR1A = 0;
    TCCR1B = 0;

    TCCR1A = (1 << WGM10);
    TCCR1B = (1 << WGM12);

    TCCR1A |= (1 << COM1A1);

    // prescaler 64
    TCCR1B |= (1 << CS11) | (1 << CS10);

    OCR1A = 0;    // start OFF
    interrupts();

```

```

    Serial.println("What LED? (1 or 2)");
}

void loop() {
    for (uint8_t i = 0; i < NUM_TASKS; i++) {
        tasks[i]();
    }
}

void TaskToggleLED1() {
    unsigned long now = millis();
    if (interval1 > 0 && now - prevMillis1 >= interval1) {
        prevMillis1 = now;
        state1 = !state1;
        digitalWrite(LED_PIN1, state1);
    }
}

void TaskToggleLED2() {
    unsigned long now = millis();
    if (interval2 > 0 && now - prevMillis2 >= interval2) {
        prevMillis2 = now;
        state2 = !state2;
        digitalWrite(LED_PIN2, state2);
    }
}

void flushSerialInput() {
    while (Serial.available()) Serial.read();
}

void TaskSerialInput() {
    if (!Serial.available()) return;

    if (step == 0) {
        int val = Serial.parseInt();
        flushSerialInput();
    }
}

```

```

    if (val == 1 || val == 2) {
        chosen_led = val;
        step = 1;
        Serial.println("What interval (msec)?");
    } else {
        Serial.println("Invalid LED. Enter 1 or 2:");
    }
}

else if (step == 1) {
    long v = Serial.parseInt();
    flushSerialInput();

    if (v > 0) {
        if (chosen_led == 1) interval1 = v;
        else interval2 = v;

        Serial.print("LED ");
        Serial.print(chosen_led);
        Serial.print(" interval set to ");
        Serial.println(v);

        step = 0;
        Serial.println("What LED? (1 or 2)");
    }
}

}

void TaskPWMBUTTON() {
    bool reading = digitalRead(BTN_PIN);

    if (reading != lastReading) lastDebounce = millis();

    if ((millis() - lastDebounce) > debounceDelay) {
        if (reading != stableState) {
            stableState = reading;

            if (stableState == LOW) { // button press

                // update brightness
            }
        }
    }
}

```

```

    brightnessIndex += direction;
    if (brightnessIndex >= 5) direction = -1;
    if (brightnessIndex <= 0) direction = +1;

    pwmDuty = map(brightnessSteps[brightnessIndex], 0, 5, 0, 255);

    OCR1A = pwmDuty;

    Serial.print("Brightness step ");
    Serial.print(brightnessIndex);
    Serial.print(" PWM = ");
    Serial.println(pwmDuty);
  }
}

lastReading = reading;
}

```

APPENDIX D. PART III DESIGN CODE

```

// PROJECT: PWM Motor Driver

// Pin assignments
#define LED_PIN1 2
#define LED_PIN2 3
#define BTN_PIN 4
#define PWM_PIN 9

#include <Arduino.h>

// CE LED
unsigned long interval1 = 0;
unsigned long interval2 = 0;
unsigned long prevMillis1 = 0;
unsigned long prevMillis2 = 0;

bool state1 = LOW;
bool state2 = LOW;

```

```

int step = 0;
int chosen_led = 0;

// PWM
volatile uint8_t pwmDuty = 0;
volatile uint8_t brightnessIndex = 0;

// Duty cycle
const uint8_t dutySteps[] = {0, 2, 4, 6, 8, 6, 4, 2};
const uint8_t NUM_STATES = 8;

// Debounce
unsigned long lastDebounce = 0;
const unsigned long debounceDelay = 40;
bool lastReading = HIGH;
bool stableState = HIGH;

//Task manager
void TaskToggleLED1();
void TaskToggleLED2();
void TaskSerialInput();
void TaskPWMBUTTON();

typedef void (*TaskFn)();
TaskFn tasks[] = {TaskToggleLED1, TaskToggleLED2, TaskSerialInput, TaskPWMBUTTON};
const uint8_t NUM_TASKS = 4;

void setup() {
    Serial.begin(9600);

    pinMode(LED_PIN1, OUTPUT);
    pinMode(LED_PIN2, OUTPUT);
    pinMode(BTN_PIN, INPUT_PULLUP);

    digitalWrite(LED_PIN1, LOW);
    digitalWrite(LED_PIN2, LOW);

    pinMode(PWM_PIN, OUTPUT);

    noInterrupts();

```

```

TCCR1A = 0;
TCCR1B = 0;

TCCR1A = (1 << WGM10);
TCCR1B = (1 << WGM12);

TCCR1A |= (1 << COM1A1);

// prescaler
TCCR1B |= (1 << CS11) | (1 << CS10);

OCR1A = 0;    // start OFF
interrupts();

Serial.println("What LED? (1 or 2)");
}

void loop() {
    for (uint8_t i = 0; i < NUM_TASKS; i++) {
        tasks[i]();
    }
}

void TaskToggleLED1() {
    unsigned long now = millis();
    if (interval1 > 0 && now - prevMillis1 >= interval1) {
        prevMillis1 = now;
        state1 = !state1;
        digitalWrite(LED_PIN1, state1);
    }
}

void TaskToggleLED2() {
    unsigned long now = millis();
    if (interval2 > 0 && now - prevMillis2 >= interval2) {
        prevMillis2 = now;
        state2 = !state2;
        digitalWrite(LED_PIN2, state2);
    }
}

void flushSerialInput() {

```

```

    while (Serial.available()) Serial.read();
}

void TaskSerialInput() {
    if (!Serial.available()) return;

    if (step == 0) {
        int val = Serial.parseInt();
        flushSerialInput();

        if (val == 1 || val == 2) {
            chosen_led = val;
            step = 1;
            Serial.println("What interval (msec)?");
        } else {
            Serial.println("Invalid LED. Enter 1 or 2:");
        }
    }

    else if (step == 1) {
        long v = Serial.parseInt();
        flushSerialInput();

        if (v > 0) {
            if (chosen_led == 1) interval1 = v;
            else interval2 = v;

            Serial.print("LED ");
            Serial.print(chosen_led);
            Serial.print(" interval set to ");
            Serial.println(v);

            step = 0;
            Serial.println("What LED? (1 or 2)");
        }
    }
}

void TaskPWMBUTTON() {
    bool reading = digitalRead(BTN_PIN);

```

```
if (reading != lastReading) lastDebounce = millis();

if ((millis() - lastDebounce) > debounceDelay) {
  if (reading != stableState) {
    stableState = reading;

    if (stableState == LOW) {      // button press

      // Step through 8-state cycle
      brightnessIndex = (brightnessIndex + 1) % NUM_STATES;

      // Convert 0-8 to 0-255 PWM
      pwmDuty = map(dutySteps[brightnessIndex], 0, 8, 0, 255);

      OCR1A = pwmDuty;

      Serial.print("Duty step ");
      Serial.print(dutySteps[brightnessIndex]);
      Serial.print("/8 PWM = ");
      Serial.println(pwmDuty);
    }
  }
}

lastReading = reading;
}
```

END OF DOCUMENT

APPENDIX B. DESIGN CODE PART I

```
//PROJECT: PWM Motor Driver (Part I)
// Pin assignments
#define LED_PIN1 2
#define LED_PIN2 3
#define BTN_PIN 4
#define PWM_PIN 9

#include <Arduino.h>

// CE LED
unsigned long interval1 = 0;
unsigned long interval2 = 0;
unsigned long prevMillis1 = 0;
unsigned long prevMillis2 = 0;

bool state1 = LOW;
bool state2 = LOW;

int step = 0;
int chosen_led = 0;
```

```

//PWM
volatile uint8_t pwmDuty = 0;
volatile uint8_t brightnessIndex = 0;

// 5 brightness steps
const uint8_t brightnessSteps[6] = {0, 1, 2, 3, 4, 5};
int direction = 1;

// Debounce
unsigned long lastDebounce = 0;
const unsigned long debounceDelay = 40;
bool lastReading = HIGH;
bool stableState = HIGH;

// Task Manager
void TaskToggleLED1();
void TaskToggleLED2();
void TaskSerialInput();
void TaskPWMBUTTON();

typedef void (*TaskFn)();
TaskFn tasks[] = {TaskToggleLED1, TaskToggleLED2, TaskSerialInput, TaskPWMBUTTON};
const uint8_t NUM_TASKS = 4;

void setup() {
    Serial.begin(9600);
    // CE LED
    pinMode(LED_PIN1, OUTPUT);
    pinMode(LED_PIN2, OUTPUT);
    //PWM BUTTON
    pinMode(BTN_PIN, INPUT_PULLUP);
    pinMode(PWM_PIN, OUTPUT);

    digitalWrite(LED_PIN1, LOW);
    digitalWrite(LED_PIN2, LOW);

    noInterrupts();
    TCCR1A = 0;
    TCCR1B = 0;

```

```

TCCR1A = (1 << WGM10);
TCCR1B = (1 << WGM12);

TCCR1A |= (1 << COM1A1);

// prescaler 64
TCCR1B |= (1 << CS11) | (1 << CS10);

OCR1A = 0;    // start OFF
interrupts();

Serial.println("What LED? (1 or 2)");
}

void loop() {
  for (uint8_t i = 0; i < NUM_TASKS; i++) {
    tasks[i]();
  }
}

void TaskToggleLED1() {
  unsigned long now = millis();
  if (interval1 > 0 && now - prevMillis1 >= interval1) {
    prevMillis1 = now;
    state1 = !state1;
    digitalWrite(LED_PIN1, state1);
  }
}

void TaskToggleLED2() {
  unsigned long now = millis();
  if (interval2 > 0 && now - prevMillis2 >= interval2) {
    prevMillis2 = now;
    state2 = !state2;
    digitalWrite(LED_PIN2, state2);
  }
}

```

```

void flushSerialInput() {
    while (Serial.available()) Serial.read();
}

void TaskSerialInput() {
    if (!Serial.available()) return;

    if (step == 0) {
        int val = Serial.parseInt();
        flushSerialInput();

        if (val == 1 || val == 2) {
            chosen_led = val;
            step = 1;
            Serial.println("What interval (msec)?");
        } else {
            Serial.println("Invalid LED. Enter 1 or 2:");
        }
    }

    else if (step == 1) {
        long v = Serial.parseInt();
        flushSerialInput();

        if (v > 0) {
            if (chosen_led == 1) interval1 = v;
            else interval2 = v;

            Serial.print("LED ");
            Serial.print(chosen_led);
            Serial.print(" interval set to ");
            Serial.println(v);

            step = 0;
            Serial.println("What LED? (1 or 2)");
        }
    }
}

void TaskPWMBUTTON() {

```

```

bool reading = digitalRead(BTN_PIN);

if (reading != lastReading) lastDebounce = millis();

if ((millis() - lastDebounce) > debounceDelay) {
    if (reading != stableState) {
        stableState = reading;

        if (stableState == LOW) { // button press

            // update brightness
            brightnessIndex += direction;
            if (brightnessIndex >= 5) direction = -1;
            if (brightnessIndex <= 0) direction = +1;

            pwmDuty = map(brightnessSteps[brightnessIndex], 0, 5, 0, 255);

            OCR1A = pwmDuty;

            Serial.print("Brightness step ");
            Serial.print(brightnessIndex);
            Serial.print(" PWM = ");
            Serial.println(pwmDuty);
        }
    }
}

lastReading = reading;
}

```